**SAVEETHA SCHOOL OF ENGINEERING**

**OPERATING SYSTEMS**

**ASSIGNMENT | CO1**

# EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

| Student Details | Information |
|---|---|
| Name | S. Maruthu Pandi |
| Register Number | 192511162 |
| Department | B.E. Computer Science and Engineering (Cyber Security) |
| Course | CSA0421 – Operating Systems |
| Assignment | Assignment 1 (Tasks 6) |

## TASK 6 – EVALUATE THE COST OF SYSTEM CALLS

### 6.1 Why a system call costs more than a normal function call

An ordinary function call usually changes the instruction pointer within the same process and privilege level. A system call requires a controlled user-to-kernel privilege transition, validation of arguments and permissions, kernel execution, and a return transition. It may also involve scheduler activity, filesystem lookup, device access, cache misses or waiting for I/O. Therefore, the transition overhead is normally much higher than a simple in-process function call.

### 6.2 Effect of buffer size

For a file containing n bytes and a read buffer of b bytes, the approximate number of successful read() calls is ceil(n/b). The program may make one additional call that returns zero to detect end-of-file. A small buffer increases the number of transitions; a larger buffer reduces call count but uses more memory. For a backup utility, a few KiB or tens/hundreds of KiB are sensible candidates to benchmark, rather than assuming that the largest buffer is always fastest.

| File size | Buffer size | Successful read calls | Including EOF read |
|---|---|---|---|
| 10,000 bytes | 1 byte | 10,000 | 10,001 |
| 10,000 bytes | 512 bytes | 20 | 21 |
| 10,000 bytes | 4,096 bytes | 3 | 4 |

The counts above are deterministic for the supplied regular-file test and the specified buffer sizes, assuming each read returns the requested amount until the final partial block. The program's own counter counts every read() attempt, including the EOF read, exactly as required by the assignment.

### 6.3 Completed C program – TODO 1

The implementation below uses read() and write() directly. It handles partial writes, retries interrupted system calls, rejects a zero or over-sized buffer and closes both file descriptors. The fixed array has 8,192 bytes, so bufsize must not exceed that capacity.

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

/* Copy src to dst using read() and write().
   Return the number of read() calls, including the EOF read, or -1. */
long copy_file(const char *src, const char *dst, size_t bufsize) {
    char buf[8192];
    if (bufsize == 0 || bufsize > sizeof buf) {
        errno = EINVAL;
        return -1;
    }

    int in = open(src, O_RDONLY);
    if (in < 0) return -1;

    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }

    long calls = 0;
    ssize_t n;
    int failed = 0;

    for (;;) {
        calls++;
        n = read(in, buf, bufsize);
        if (n == 0) break;
        if (n < 0) {
            if (errno == EINTR) continue;
            failed = 1;
            break;
        }
```

```
        /* A write() may write fewer than n bytes. */
        ssize_t written = 0;
        while (written < n) {
            ssize_t w = write(out, buf + written, (size_t)(n - written));
            if (w < 0) {
                if (errno == EINTR) continue;
                failed = 1;
                break;
            }
            if (w == 0) {
                errno = EIO;
                failed = 1;
                break;
            }
            written += w;
        }
        if (failed) break;
    }

    int saved_errno = errno;
    if (close(in) < 0 && !failed) { failed = 1; saved_errno = errno; }
    if (close(out) < 0 && !failed) { failed = 1; saved_errno = errno; }
    if (failed) { errno = saved_errno; return -1; }
    return calls;
}

int main(void) {
    FILE *f = fopen("records.dat", "w");
    if (f == NULL) { perror("fopen"); return 1; }
    for (int i = 0; i < 10000; i++) {
        if (fputc('A' + i % 26, f) == EOF) {
            perror("fputc"); fclose(f); return 1;
        }
```

```
    }
    if (fclose(f) == EOF) { perror("fclose"); return 1; }

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++) {
        long calls = copy_file("records.dat", "copy.dat", sizes[i]);
        if (calls < 0) { perror("copy_file"); return 1; }
        printf("buffer %4zu bytes: %5ld read() calls\n",
               sizes[i], calls);
    }

    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

6.4 Compile and run

```
gcc -std=c11 -Wall -Wextra -O2 task06_copy.c -o task06_copy
./task06_copy
strace -c -e trace=read,write,openat,close ./task06_copy
```

6.5 Expected output

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

The assignment's expected output is reproduced. The output is the program's own count for read() calls inside copy_file(). A strace -c summary can show additional calls made by program startup, the C library, file creation and printing. Use strace -e trace=read -o task06.trace ./task06_copy to inspect individual calls and compare the copy loop with the program counter.

6.6 Buffer-size recommendation

For the supplied 10,000-byte file, 4,096 bytes reduces the read-call count from 10,001 (1-byte buffer) to 4 (4,096-byte buffer). That is a major reduction in syscall overhead with only a 4

KiB buffer. For a production hospital backup, test 4 KiB, 64 KiB, 256 KiB and 1 MiB using representative files and storage. Choose the buffer that gives the best repeatable throughput without excessive memory use or unacceptable latency for competing workloads. The exact optimum must be measured on the actual server.

### 6.7 Alternative: memory-mapped files (mmap)

With mmap(), the program maps file contents into its virtual address space and accesses them as memory. A simple mapped copy can use open(), fstat(), mmap() for the source and destination, memcpy(), msync() when required, munmap() and close(). This can reduce explicit read()/write() calls from roughly n/b to a small number of mapping calls. It does not eliminate kernel work: page faults, page-cache activity and dirty-page writeback still occur. The copy still moves O(n) bytes, and memcpy() still consumes memory bandwidth.

For a very large file, mapping the entire file uses a large virtual address range and may create resource-management issues. A windowed mapping approach maps a bounded portion at a time, copies it, then unmaps it. The program must handle mapping errors, file-size changes, alignment requirements and destination sizing. mmap() is a useful alternative for some workloads, but buffered read()/write() can be simpler and more predictable for a sequential backup utility